Semana 13 — Guía de Práctica

Diseño de Interfaces Gráficas (GUI) con Tkinter

Construcción de Software · Universidad Continental

Ing. Henderson Nozart Gibaja Reyes

1. Objetivo

Al finalizar esta práctica, el estudiante será capaz de diseñar e implementar interfaces gráficas de usuario (GUI) utilizando Python Tkinter, aplicando principios de usabilidad y accesibilidad para crear aplicaciones de escritorio funcionales con validación de datos.

2. Conceptos clave

- GUI (Graphical User Interface): Interfaz visual que permite al usuario interactuar con el software mediante elementos gráficos.
- Tkinter: Biblioteca estándar de Python para crear interfaces gráficas. Viene incluida con Python, no requiere instalación.
- Widget: Componente visual de la interfaz (botón, campo de texto, etiqueta, etc.).
- Layout Manager: Sistema de posicionamiento de widgets (pack, grid, place).
- Evento: Acción del usuario (click, tecla, etc.) que dispara una función.
- Wireframe: Boceto de baja fidelidad que define la estructura de la interfaz.
- Usabilidad: Facilidad con la que un usuario puede usar la interfaz para lograr su objetivo.
- Accesibilidad: Capacidad de la interfaz para ser utilizada por todos los usuarios, incluyendo personas con discapacidades.

3. Preparación del entorno

Tkinter viene incluido con Python. Verifiquemos que está disponible:

```
python -c "import tkinter; print(tkinter.TkVersion)"
```

Si muestra un número de versión (ej: 8.6), Tkinter está instalado correctamente.

Crear la estructura del proyecto:

```
mkdir semana13-gui
cd semana13-gui
```

4. Ejercicio 1: Calculadora básica con GUI

Vamos a construir una calculadora paso a paso, aplicando el diseño antes del código.

Paso 1: Diseñar el wireframe

Antes de escribir código, dibuja un wireframe (en papel o herramienta digital) con:

- Un campo de texto (display) en la parte superior
- 4 filas de 4 botones cada una (números y operadores)
- Botón C para limpiar y = para calcular

Paso 2: Crear la ventana principal

```
import tkinter as tk
from tkinter import messagebox

class Calculadora:
    def __init__(self):
        self.ventana = tk.Tk()
        self.ventana.title("Calculadora - Semana 13")
        self.ventana.geometry("300x400")
        self.ventana.resizable(False, False)

        # Display
        self.display = tk.Entry(
            self.ventana,
            font=("Arial", 24),
            justify="right",
            bd=5
        )
        self.display.grid(
            row=0, column=0, columnspan=4,
            sticky="nsew", padx=5, pady=5
        )
```

Paso 3: Agregar los botones

```
        # Definir botones en una matriz
        botones = [
            ('7', '8', '9', '/'),
            ('4', '5', '6', '*'),
            ('1', '2', '3', '-'),
            ('C', '0', '=', '+'),
        ]

        for i, fila in enumerate(botones):
            for j, texto in enumerate(fila):
                btn = tk.Button(
                    self.ventana,
```

```
                    text=texto,
                    font=("Arial", 18),
                    width=4,
                    height=2,
                    command=lambda t=texto: self.click(t)
                )
                btn.grid(row=i+1, column=j, padx=2, pady=2)
```

Paso 4: Implementar la lógica de eventos

```
    def click(self, valor):
        if valor == 'C':
            self.display.delete(0, tk.END)
        elif valor == '=':
            try:
                expresion = self.display.get()
                resultado = eval(expresion)
                self.display.delete(0, tk.END)
                self.display.insert(0, str(resultado))
            except Exception:
                messagebox.showerror("Error", "Expresión inválida")
                self.display.delete(0, tk.END)
        else:
            self.display.insert(tk.END, valor)

    def run(self):
        self.ventana.mainloop()

if __name__ == "__main__":
    Calculadora().run()
```

Paso 5: Ejecutar y probar

```
python calculadora.py
```

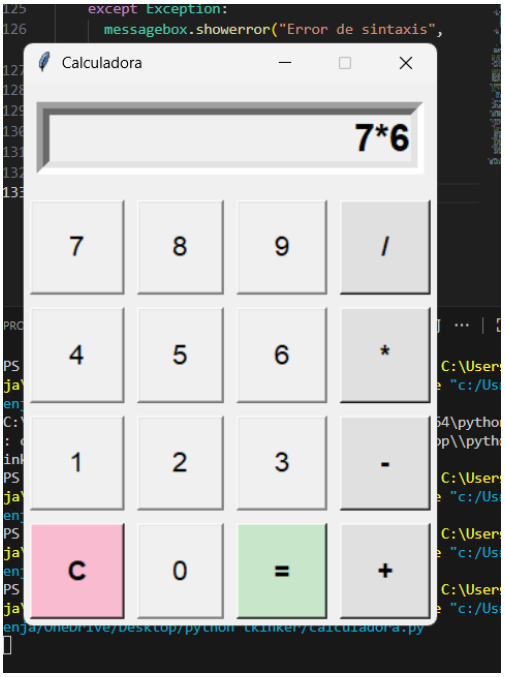
Prueba las siguientes operaciones:

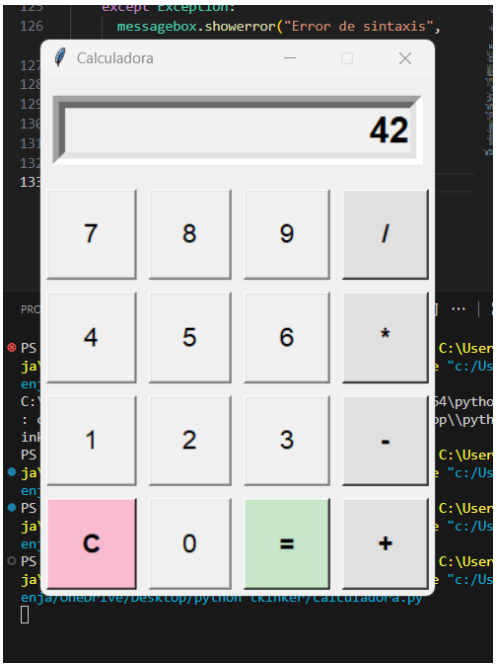
- Números normales: 5+3 → 8





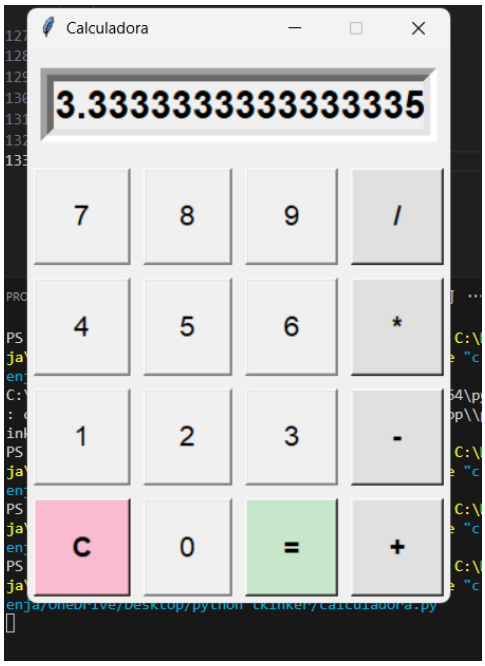
- Multiplicación: 7*6 → 42





- División: 10/3 → 3.333...





-
- Limpiar con C y volver a calcular
  si limpia
- Expresión inválida: ++ → mensaje de error

  No permite poner ++ ni dividir entre 0

5. Ejercicio 2: Formulario de registro con validación

Este ejercicio aplica todos los conceptos de usabilidad y accesibilidad.

Paso 1: Diseñar el wireframe

Dibuja un formulario con estos campos (usa papel o Excalidraw):

- Nombre completo (campo de texto)
- Email (campo de texto)
- Contraseña y confirmar contraseña (campos ocultos)
- Edad (campo numérico)
- Género (radio buttons: Masculino, Femenino, Otro)
- Acepto términos (checkbox)
- Botones: Registrar y Limpiar

Paso 2: Crear la estructura base

```
import tkinter as tk
from tkinter import messagebox
import re

class FormularioRegistro:
```

```
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Registro de Usuario")
        self.root.geometry("500x600")

        # Variables de los campos
        self.nombre = tk.StringVar()
        self.email = tk.StringVar()
        self.password = tk.StringVar()
        self.confirm_pass = tk.StringVar()
        self.edad = tk.StringVar()
        self.genero = tk.StringVar(value="")
        self.acepto = tk.BooleanVar()

        self.crear_widgets()

    def crear_widgets(self):
        # Frame principal con padding
        frame = tk.Frame(self.root, padx=20, pady=20)
        frame.pack(fill="both", expand=True)

        # Título
        tk.Label(
            frame, text="Registro de Usuario",
            font=("Arial", 16, "bold")
        ).grid(row=0, column=0, columnspan=2, pady=(0, 15))

        # Campos de texto
        campos = [
            ("Nombre completo:", self.nombre, False),
            ("Email:", self.email, False),
            ("Contraseña:", self.password, True),
            ("Confirmar contraseña:", self.confirm_pass, True),
            ("Edad:", self.edad, False),
        ]

        for i, (label, var, oculto) in enumerate(campos):
            tk.Label(frame, text=label, anchor="e").grid(
                row=i+1, column=0, sticky="e", pady=5, padx=(0,10)
            )
            entry = tk.Entry(
                frame, textvariable=var, width=35,
                show="*" if oculto else ""
            )
            entry.grid(row=i+1, column=1, pady=5, sticky="w")
```

Paso 3: Agregar radio buttons, checkbox y botones

```
        # Género (radio buttons)
        tk.Label(frame, text="Género:").grid(
            row=6, column=0, sticky="e", pady=5
        )
        genero_frame = tk.Frame(frame)
        genero_frame.grid(row=6, column=1, sticky="w")
        for opcion in ["Masculino", "Femenino", "Otro"]:
            tk.Radiobutton(
                genero_frame, text=opcion,
                variable=self.genero, value=opcion
            ).pack(side="left", padx=5)

        # Checkbox de términos
        tk.Checkbutton(
            frame, text="Acepto los términos y condiciones",
            variable=self.acepto
        ).grid(row=7, column=0, columnspan=2, pady=10)

        # Botones
        btn_frame = tk.Frame(frame)
        btn_frame.grid(row=8, column=0, columnspan=2, pady=15)
        tk.Button(
            btn_frame, text="Registrar",
            command=self.validar, width=15,
            bg="#4CAF50", fg="white"
        ).pack(side="left", padx=10)
        tk.Button(
            btn_frame, text="Limpiar",
            command=self.limpiar, width=15
        ).pack(side="left", padx=10)
```

Paso 4: Implementar la validación

```
    def validar(self):
        errores = []

        # Nombre obligatorio
        if not self.nombre.get().strip():
            errores.append("El nombre es obligatorio")

        # Email con formato válido
        email = self.email.get().strip()
        if not re.match(r"[^@]+@[^@]+\.[^@]+", email):
            errores.append("Email inválido (debe contener @ y dominio)")

        # Contraseña mínimo 8 caracteres
        if len(self.password.get()) < 8:
            errores.append("La contraseña debe tener mínimo 8 caracteres")
```

```
        # Contraseñas coinciden
        if self.password.get() != self.confirm_pass.get():
            errores.append("Las contraseñas no coinciden")

        # Edad válida
        try:
            edad = int(self.edad.get())
            if edad < 15 or edad > 99:
                errores.append("La edad debe estar entre 15 y 99")
        except ValueError:
            errores.append("La edad debe ser un número")

        # Género seleccionado
        if not self.genero.get():
            errores.append("Seleccione un género")

        # Términos aceptados
        if not self.acepto.get():
            errores.append("Debe aceptar los términos")

        if errores:
            messagebox.showerror(
                "Errores de validación",
                "\n".join(f"• {e}" for e in errores)
            )
        else:
            messagebox.showinfo(
                "Registro exitoso",
                f"¡Bienvenido {self.nombre.get()}!\n"
                f"Email: {self.email.get()}"
            )

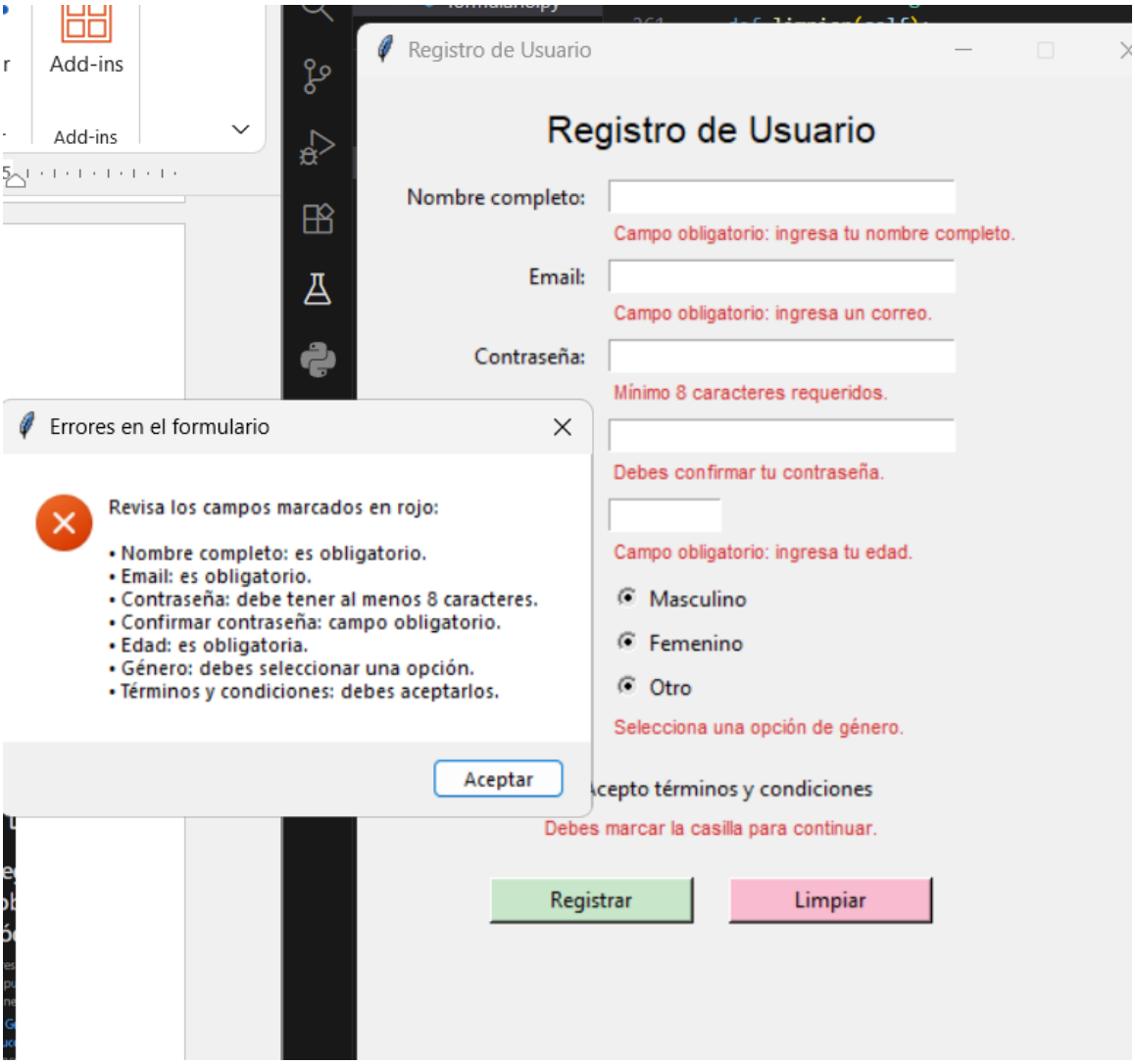
    def limpiar(self):
        for var in [self.nombre, self.email, self.password,
                    self.confirm_pass, self.edad, self.genero]:
            var.set("")
        self.acepto.set(False)

if __name__ == "__main__":
    app = FormularioRegistro()
    app.root.mainloop()
```

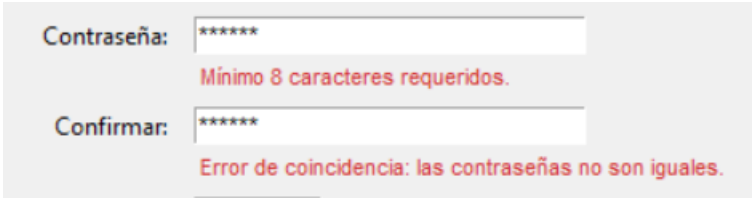
Paso 5: Ejecutar y probar todas las validaciones

```
python formulario.py
```

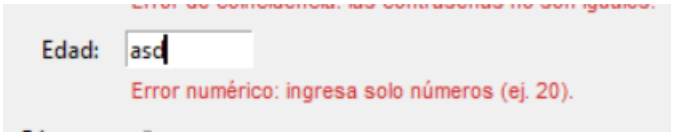
Prueba los siguientes casos:

- Enviar con todos los campos vacíos → debe mostrar todos los errores


- Email sin @ → error de formato


- Contraseñas diferentes → error de coincidencia


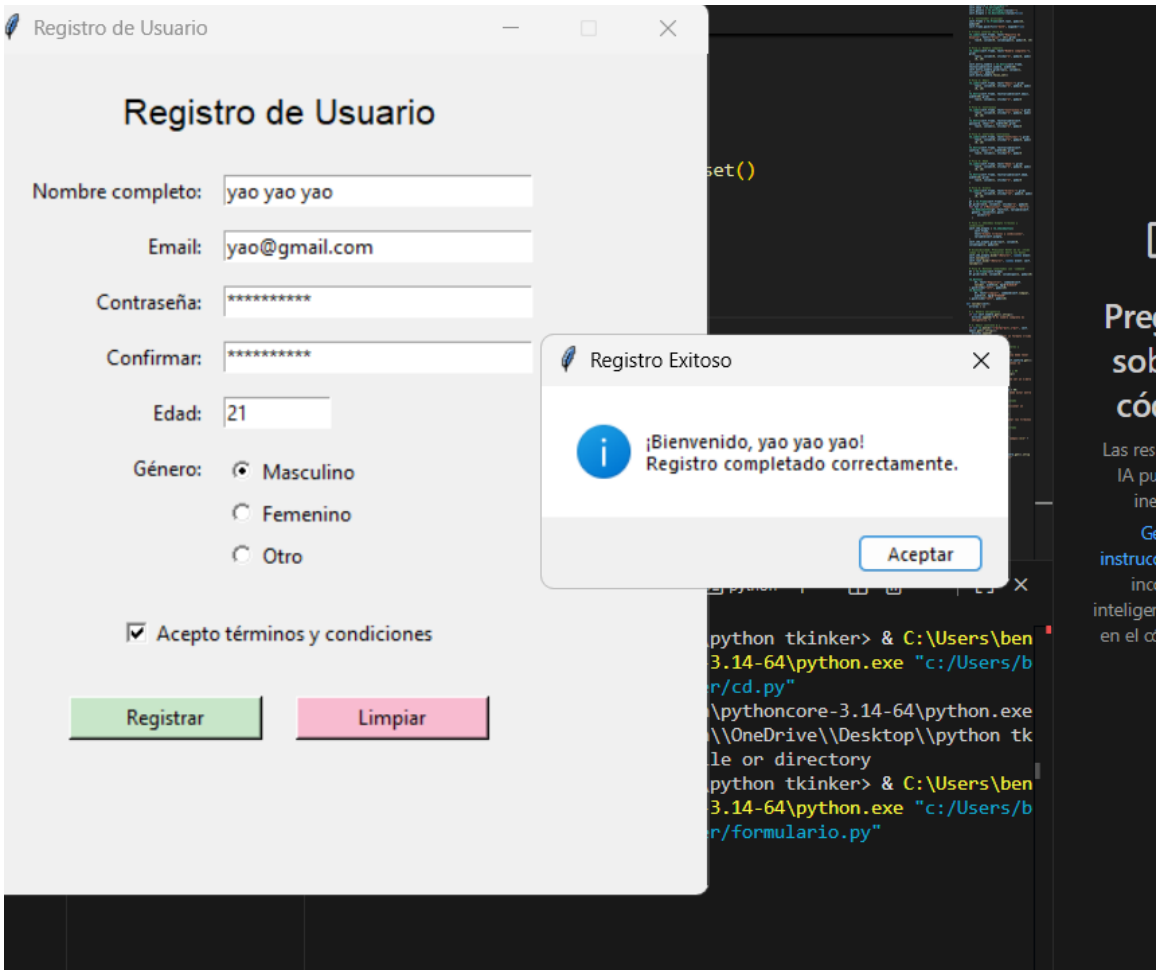
- Edad "abc" → error numérico



- Todo correcto → mensaje de bienvenida
- Botón Limpiar → todos los campos se resetean

Si resetea

En cualquiera de los casos de error de formato da ese error



6. Entregable

Subir al repositorio GitHub del equipo:

Benjaminsalas-Design/formulario-

- calculadora.py — Calculadora funcional con GUI
- formulario.py — Formulario de registro con validaciones
- wireframe.png — Boceto/wireframe del formulario (foto de papel o herramienta digital)
- README.md — Explicación del diseño: qué principios de usabilidad aplicaron y por qué
- Capturas de pantalla de la aplicación funcionando

7. Rúbrica de evaluación

- Wireframe del formulario: 2 pts — Incluye wireframe claro con todos los elementos, bien organizado
- Calculadora funcional: 3 pts — Calculadora con todas las operaciones, manejo de errores, interfaz limpia
- Formulario con validación: 4 pts — Todos los campos, todas las validaciones, mensajes de error claros
- Usabilidad y accesibilidad: 3 pts — Labels descriptivos, tab order, atajos de teclado, retroalimentación visual
- Organización del código: 2 pts — Uso de clases, código limpio, separación de lógica e interfaz
- README.md: 1 pt — Explica decisiones de diseño y principios aplicados
- Commits incrementales: 1 pt — Al menos 5 commits que muestren progreso

Total: 16 puntos